

Undefined Topic

Explain why you would choose a LinkedHashSet (or LinkedHashMap) over a HashSet (or HashMap, respectively). [2]

ANS:

HashSet does not provide any method to maintain the insertion order. Comparatively, LinkedHashSet maintains the insertion order of the elements. We can not predict the insertion order in HashSet, but we can predict it in LinkedHashSet. The LinkedHashSet extends the HashSet, so it uses a hashtable to store the elements.

<https://www.geeksforgeeks.org/difference-and-similarities-between-hashset-linkedhashset-and-treeset-in-java/>

The Collections class of Java Collection Framework has several methods (e.g. sort(), min(), max()) that require natural ordering of the classes to be handled. Explain the two possible conditions that must be satisfied in the classes to be able to use the sort() method to arrange the list of objects these classes. [2]

ANS:

In Java, the collections framework provides a static method sort() that can be used to sort elements in a collection.

The `sort()` method of the collections framework uses the merge sort algorithm to sort elements of a collection.

The merge sort algorithm is based on divide and conquers rule. To learn more about the merge sort, visit Merge Sort Algorithm.

What is dangling-else problem? [1]

ANS:

The dangling else problem in syntactic ambiguity. It occurs when we use *nested if*. When there are multiple “if” statements, the “else” part doesn’t get a clear view with which “if” it should combine.

For example:

```

if (condition) {
}
if (condition 1) {
}
    if (condition 2) {
    }
    else
    {
        }

```

In the above example, there are multiple “*ifs*” with multiple conditions and here we want to pair the outermost if with the else part. But the else part doesn’t get a clear view with which “*if*” condition it should pair. This leads to inappropriate results in programming.

What is a package? How can you define your own package? [3]

ANS: Package in Java is a mechanism to encapsulate a group of classes, sub packages and interfaces. Packages are used for: Preventing naming conflicts. For example there can be two classes with name Employee in two packages, college.

How to create package in Java

1. First create a directory within name of package.
2. Create a java file in newly created directory.
3. In this java file you must specify the package name with the help of package keyword.
4. Save this file with same name of public class. ...
5. Now you can use this package in your program.

#> Pascal and C are ____ languages, whereas C++ is object-oriented language.

Ans: procedural

OOP

Object-oriented programming is about creating objects that contain both data and methods.

Three Fundamental Aspects of OOP

The three fundamental features of object-oriented programming are **encapsulation**, **inheritance**, and **polymorphism**.

1. **Encapsulation** promotes the concept of **information hiding**, which is useful because it protects information from being changed or altered by other parts of a program.
2. **Inheritance** allows one to **reuse existing software** so it allows significant improvement in productivity.
3. **Polymorphism** allows data types and functions to **belong to more generic classes**.

#> Which of the following approach is adapted by OOP Concept?

Ans: Bottom-Up

Why java oop language has become popular?

- Java is **user-friendly**
- Java for **everything!**
- Java boasts of **rich API**
- A robust **community** backs Java
- Java has **excellent and free documentation**
- Java has a suite of **powerful development tools**
- An **Omnipresent** (সর্বব্যাপী)

(As per the latest stats, there are more than 3 billion devices that are running primarily on Java.

Android apps, web applications, software tools such as Eclipse, IntelliJ IDEA, and NetBeans IDE, scientific applications such as natural language processing are all being developed in Java.)

- Very Important language for the technology such that **IoT, Machine Learning, and Data Science**
- Widely **Used by Industry Giants**

(Ranging from Macintosh, Unix/Linux, mainframe systems, Windows and mobile phones, Java's RunTime Environment (JRE) is beautifully compatible with any existing device.)

What are the pros and cons of OOP? Pros:

1. Improved software development productivity
2. Improved software maintainability
3. Faster development
4. Lower cost of development
5. Higher quality software
6. Allows parallel development
7. Modular classes are often reusable
8. Coding is easier to maintain

Cons:

1. Larger program size
2. It can be inefficient
3. It can cause duplication
4. It can be too scalable

Details:

extension://bfdogplmndidlpjfhiojckpakkdjkkil/pdf/viewer.html?file=https%3A%2F%2Fresources.saylor.org%2Fwwwresources%2Farchived%2Fsite%2Fwp-content%2Fuploads%2F2013%2F02%2FCS101-2.1.2-AdvantagesDisadvantagesOfOOP-FINAL.pdf

[6 Pros and Cons of Object Oriented Programming – Green Garage \(greengarageblog.org\)](https://www.greengarageblog.org/6-pros-and-cons-of-object-oriented-programming/)

What is object-oriented programming?

Object-oriented programming is a programming paradigm based on the concept of "objects", which can contain data and code: data in the form of fields, and code, in the form of procedures.

Object-oriented programming is an approach that provides a way of modularizing programs by creating partitioned memory areas for both data and functions that can be used as templates for creating copies of such modules on demand.

Fundamental Concepts of OOP

Class, Attribute, Method, Object, Encapsulation, Inheritance, Abstraction, Polymorphism

What is structured programming language?

Structured programming (sometimes known as *modular programming*) is a programming paradigm that facilitates the creation of programs with **readable code and reusable components**. All modern programming languages support structured programming, but the mechanisms of support, like the [syntax](#) of the programming languages, varies.

What is unstructured programming language?

Unstructured Programming is a type of programming that generally executes in sequential order i.e., these programs just not jumped from any line of code and each line gets executed sequentially.

Unstructured programming is a **procedural program** – the statements are executed in sequence as written(The statements execute in order you write)

In this you have to use goto statements that allow the control to pass. When a goto statement is executed, the sequence continues from the target of the goto. Thus to understand how a program works, you have to pretend to execute it. This means that it is often difficult to understand the logic of such a program

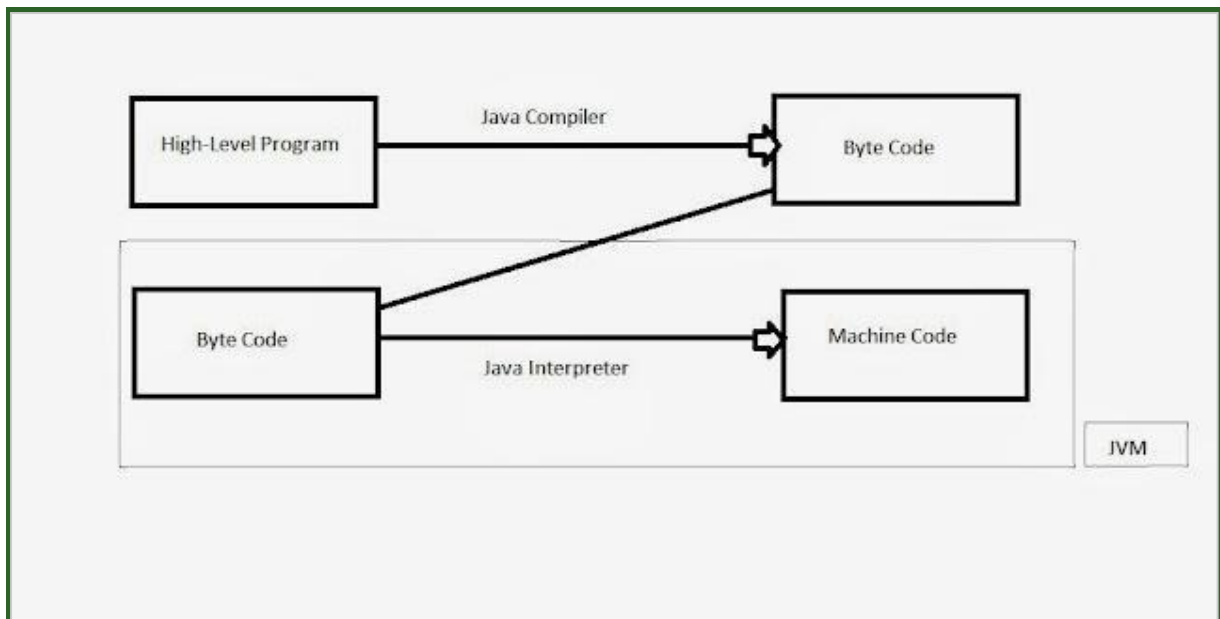
Java

Why is java known as a platform-neutral language? [2]

Platform neutral or platform independence, means that programs written in the Java language must run similarly on any supported hardware/operating-system platform. A programmer should be able to write a program one time, compile it one time, and then be able to execute it anywhere; holding true to the Sun Microsystems slogan, "Write Once, Run Anywhere."

Java is known as platform independent because it uses the concept of generating the byte code of the high level program ,and then run that byte code(intermediate code) on JVM(Java Virtual Machine),now what is JVM,actual JVM is a virtual computer system that run on your original computer system .

JVM converts the byte code to machine code according to the your original computer's machine architecture(your computer system architecture like **x86,ARM** etc.).Here is flow how the Java handle your high level program.



So java compilation is done only once ,after that the byte code can be interpreted on any machine that have JVM.JVM is of different type according to computer system architecture,means for x86 JVM will be different for ARM JVM will be different etc.These are developed by the java vender that is **Oracle**.This JVM come as a part of JDK(Java Development Kit). So this compiler and interpreter has make it platform-neutral language.

What are the basic steps in writing a JAVA application? [2]

1. Install Java development Kit (JDK)
2. Set the path of JDK
3. Write a sample Java program. i) Open up Notepad ii) Write class name
4. Set class visibility
5. Write main execution method
6. Write print statement
7. Save Notepad
8. Compile the program
9. Run the program

Details: [How to Write a Simple Java Program - Step by Step Procedure with Illustration \(pediaa.com\)](https://www.pediaa.com/how-to-write-a-simple-java-program-step-by-step-procedure-with-illustration/)

What do you know about Java garbage collection and finalize method? Explain briefly. [3]

Garbage Collection:

In java, garbage means **unreferenced objects**.

Garbage Collection is process of reclaiming the runtime unused memory automatically. In other words, it is a way to destroy the unused objects.

To do so, we were using `free()` function in C language and `delete()` in C++. But, in java it is performed automatically. So, java provides better memory management.

finalize Method:

Finalization is a feature of the Java programming language that allows you to perform postmortem cleanup on objects that the garbage collector has found to be unreachable. It is typically used to reclaim native resources associated with an object.

`Finalize()` is the method of `Object` class. This method is called just before an object is garbage collected.

The purpose of a `finalize()` method can be overridden for an object to include the cleanup code or to dispose of the system resources that can be done before the object is garbage collected.

!# Explain the keyword `super` and function `super()` in Java with example. [3]

super keyword:

The `super` keyword refers to superclass (parent) objects.

It is used to call superclass methods, and to access the superclass constructor.

The most common use of the `super` keyword is to eliminate the confusion between superclasses and subclasses that have methods with the same name.

super() function:

The `super()` function is used to give access to methods and properties of a parent or sibling class. The `super()` function returns an object that represents the parent class.

What are different access specifiers in Java? Explain with example. [3]

[Access specifiers for classes or interfaces in Java - GeeksforGeeks](#)

Why using procedural languages like c in building large-scale applications are so uncomfortable and unmanageable?

Procedural Programming: Procedural Programming may be the first programming paradigm that a new developer will learn. Fundamentally, the procedural code is the one that directly instructs a device on how to finish a task in logical steps

Disadvantages of Procedural Programming:

A major disadvantage of using Procedural Programming as a method of programming is the inability to reuse code throughout the program. Having to rewrite the same type of code many times throughout a program can add to the development cost and time of a project. Another disadvantage is the difficulty in error-checking.

The program code is harder to write when Procedural Programming is employed

Difficult to relate with real-world objects

The importance is given to the operation rather than the data, which might pose issues in some data-sensitive cases

The data is exposed to the whole program, making it not so much security friendly

How does java compare to C++ and other OOP languages?

- C++ is both procedural and object-oriented programming language whereas Java is a pure object-oriented language.
- Both the languages have different objectives which means it has many differences too.
- The main objective of C++ is to design a system of programming

Differences

Objects and Classes [1]

[Difference between Class and Object - GeeksforGeeks](#)

Data Abstraction and Data Encapsulation [1]

Data Abstraction can be described as the technique of hiding internal details of a program and exposing the functionality only.	Data Encapsulation can be described as the technique of binding up of data along with its correlate methods as a single unit.
Implementation hiding is done using this technique.	Information hiding is done using this technique.
Data abstraction can be performed using Interface and an abstract class.	Data encapsulation can be performed using the different access modifiers like protected, private, and packages.
Abstraction lets somebody see the What part of program purpose.	Encapsulation conceals or covers the How part of the program's purpose.

<https://www.w3schools.in/java-questions-answers/difference-between-data-abstraction-and-data-encapsulation-in-java/>

Inheritance and Polymorphism [1]

S.N	Inheritance	Polymorphism
O		
1.	Inheritance is one in which a new class is created (derived class) that inherits the features from the already existing class(Base class).	Whereas polymorphism is that which can be defined in multiple forms.
2.	It is basically applied to classes.	Whereas it is basically applied to functions or methods.
3.	Inheritance supports the concept of reusability and reduces code length in object-oriented programming.	Polymorphism allows the object to decide which form of the function to implement at compile-time (overloading) as well as run-time (overriding).
4.	Inheritance can be single, hybrid, multiple, hierarchical and multilevel inheritance.	Whereas it can be compiled-time polymorphism (overload) as well as run-time polymorphism (overriding).

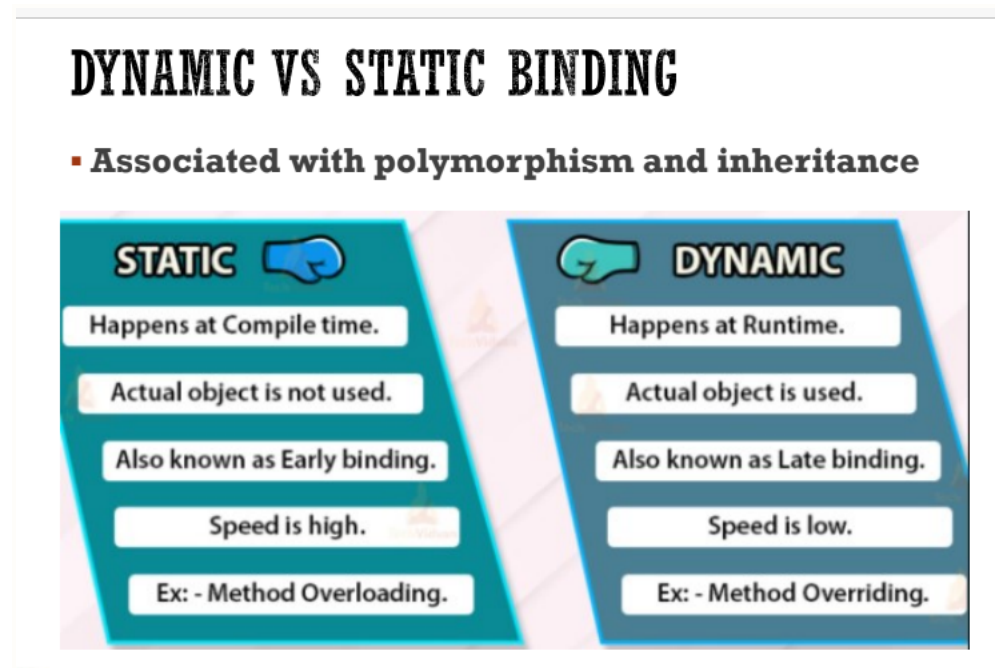
5. It is used in pattern designing. While it is also used in pattern designing.

<https://www.geeksforgeeks.org/difference-between-inheritance-and-polymorphism/>

Dynamic Binding and Message Passing [1]

<https://www.geeksforgeeks.org/differences-between-dynamic-binding-and-message-passing-in-java/>

Also:



Abstraction & Encapsulation

Here is the main difference between Encapsulation and Abstraction:



Difference between Abstraction and Encapsulation in Java

Parameter	Abstraction	Encapsulation
Use for	Abstraction solves the problem and issues that arise at the design stage.	Encapsulation solves the problem and issue that arise at the implementation stage.
Focus	Abstraction allows you to focus on what the object does instead of how it does it	Encapsulation enables you to hide the code and data into a single unit to secure the data from the outside world.
Implementation	You can use abstraction using Interface and Abstract Class.	You can implement encapsulation using Access Modifiers (Public, Protected & Private.)
Focuses	Focus mainly on what should be done.	Focus primarily on how it should be done.
Application	During design level.	During the Implementation level.

InputStream and Reader Classes [2]

InputStreams are used to read bytes from a stream . It grabs the data byte by byte without performing any kind of translation. So they are useful for binary data such as images, video and serialized objects.

Readers on the other hand are character streams so they are best used to read character data. If the information you are reading is all text, then the Reader will take care of the character decoding for you and give you unicode characters from the raw input stream. If you are reading any type of text,

this is the stream to use

<https://www.javamadesoeasy.com/2015/08/difference-between-stream.html>

OutputStream and Writer Classes [2]

- An OutputStream is a stream that can write information. This is fairly general, so there are specialized OutputStream for special purposes like writing to files. A stream can only write arrays of bytes.

Writer s provide more flexibility in that they can write characters and even strings while taking a special encoding into account.

- OutputStream classes writes to the target byte by byte where as Writer classes writes to the target character by character
- The Writer class hierarchy is character-oriented, and the Output Stream class hierarchy is byte-oriented.

break and continue keyword [3]

<https://www.tutorialspoint.com/difference-between-continue-and-break-statements-in-java>

method overloading and method overriding [3]

Difference Between Method Overloading and Method Overriding in Java

Parameter	Method Overloading in Java	Method Overriding in Java
Type of Argument	The argument type needs to be different in Method Overloading (at least the order).	The argument type needs to be the same in Method Overriding (including ` the order).

Return Type	It can be different or the same in this case. But it is a must for a user to change the parameter.	It must be the very same until the 1.4 version of Java only. After that, it only allows the Covariant return type from Java 1.5 onwards.
Access Modifiers	You can use any access modifier, or it can be different.	The access modifier for a subclass method must be the very same or higher than the access modifier of the superclass method.
Method Signatures	Every method signature must be different (with the same name) in the case of Method Overloading.	Every method signature must be the same (with the same name) in the case of Method Overriding.
Class	A user can generally perform method overloading within the same class.	A user can usually perform the method overriding in two of the classes through the Inheritance (considered an Is-A relationship).
Final/Static/Private Method	A user can easily overload it.	It is not possible for a user to override it.
Method Resolution	A user can always take care of it with a Java compiler based on the reference type.	A user can always take care of it with the JVM based on the runtime object.

Performance	The method overloading exhibits much better performance.	The method overriding usually exhibits a lesser performance.
Polymorphism	It is also known as the early binding, static polymorphism, or compile-time polymorphism.	It is also known as late binding, dynamic polymorphism, or runtime polymorphism.
Uses	The method overloading assists in raising the program's readability.	It assists in granting the specific implementation of any method (that the superclass or parent class provides).
Inheritance	It may or may not be requiring inheritance.	It is always in need of inheritance.
Parameter	The parameter needs to be different in the case of method overloading.	The parameter needs to be the same in the case of method overriding.

Also:

OVERRIDING AND OVERLOADING

Overriding

```

class Dog{
    public void bark(){
        System.out.println("woof ");
    }
}
class Hound extends Dog{
    public void sniff(){
        System.out.println("sniff ");
    }

    public void bark(){
        System.out.println("bowl");
    }
}
  
```

Same Method Name,
Same parameter

Overloading

```

class Dog{
    public void bark(){
        System.out.println("woof ");
    }

    //overloading method
    public void bark(int num){
        for(int i=0; i<num; i++)
            System.out.println("woof ");
    }
}
  
```

Same Method Name,
Different Parameter

abstract class and interface [3]

Difference between Abstract Class and Interface in Java

S.No.	Abstract Class	Interface
1.	An abstract class can contain both abstract and non-abstract methods.	Interface contains only abstract methods.
2.	An abstract class can have all four; static, non-static and final, non-final variables.	Only final and static variables are used.
3.	To declare abstract class abstract keywords are used.	The interface can be declared with the interface keyword.
4.	It doesn't support multiple inheritance.	It supports multiple inheritance.

5.	The keyword 'extend' is used to extend an abstract class	The keyword implement is used to implement the interface.
6.	It has class members like private and protected, etc.	It has class members public by default.

final vs finally vs finalize

Final is a keyword and is used as access modifier in Java. Finally is a block in Java used forethod in Java used for Garbage Collection. Exception Handling. Finalize is a m

<https://techdifferences.com/difference-between-final-finally-and-finalize-in-java.html#:~:text=The%20basic%20difference%20between%20final%2C%20finally%2C%20and%20finalize,finalize%20which%20are%20discussed%20in%20the%20comparison%20chart.>

Abstraction vs Encaptulation

Abstraction	Encapsulation
Abstraction is the process or method of gaining information.	While encapsulation is the process or method to contain the information.
In abstraction, problems are solved at the design or interface level.	While in encapsulation, problems are solved at the implementation level.
Abstraction is the method of hiding unwanted information.	Whereas encapsulation is a method to hide the data in a single entity or unit along with a method to protect information from outside.

We can implement abstraction using abstract class and interfaces.	Whereas encapsulation can be implemented using by access modifier i.e. private, protected, and public.
In abstraction, implementation complexities are hidden using abstract classes and interfaces.	While in encapsulation, the data is hidden using methods of getters and setters.

##inputstream & outputstream

<https://www.geeksforgeeks.org/difference-between-inputstream-and-outputstream-in-java/>

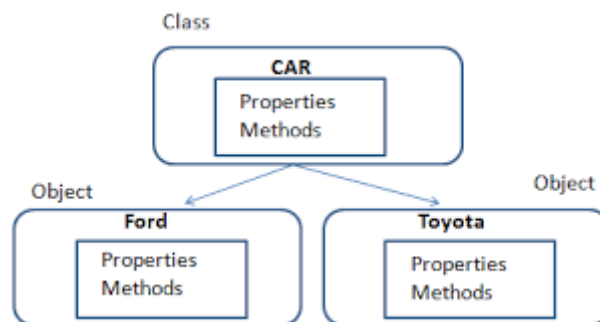
procedural programming & oop

<https://askanydifference.com/difference-between-object-oriented-programming-and-procedural-programming-with-table/#:~:text=The%20main%20difference%20between%20object-oriented%20and%20procedural%20programming,approach%2C%20whereas%20procedural%20programming%20is%20a%20top-down%20approach.>

Class

What is class? Briefly explain with an example.

Class are a blueprint or a set of instructions to build a specific type of object. It is a basic concept of Object-Oriented Programming which revolve around real-life entities. Class in Java determines how an object will behave and what the object will contain.



Syntax of Class in Java

```
class <class_name>{
    field;
    method;
}
```

//

Create a class named "Main" with a variable x:

```
public class Main {
```

```
int x = 5;
}
//
```

#> Draw the class diagram that shows inheritance, association, aggression, composition relationship between classes for a real-world scenario for a banking application. [10]

#> Which of the following term is used for a function defined inside a class?

Ans: Member Function

#> Why do you know about the 'class' concept? What is the purpose of a class?

A perfect analogy of class is a blueprint.

A blueprint is a sketch prepared before a house is actually built. It might have specifications about the house. A house built using that blueprint is an instance of that blueprint.

Similarly, an object is an instance of a class. The class definition would specify variables and behaviors in the form of member variables and methods. When an instance is created the variables contain the state of the object and methods can represent or alter the state of an object.

Explain how the Scanner class can be used to take input from the user with an example. [3]

Scanner is a class in java.util package used for obtaining the input of the primitive types like int, double, etc., and strings. It is the easiest way to read input in a Java program, though not very efficient if you want an input method for scenarios where time is a constraint like in competitive programming.

- To create an object of the Scanner class, we usually pass the predefined object System.in, which represents the standard input stream. We may pass an object of class File if we want to read input from a file.
- To read numerical values of a certain data type XYZ, the function to use is nextXYZ(). For example, to read a value of type short, we can use nextShort()
- To read strings, we use nextLine().
- To read a single character, we use next().charAt(0). next() function returns the next token/word in the input as a string and charAt(0) function returns the first character in that string.

Let us look at the code snippet to read data of various data types.

// Java program to read data of various types using Scanner class.

```
import java.util.Scanner;

public class ScannerDemo1
{
    public static void main(String[] args)
    {
        // Declare the object and initialize with
        // predefined standard input object
        Scanner sc = new Scanner(System.in);

        // String input
        String name = sc.nextLine();

        // Character input
        char gender = sc.next().charAt(0);

        // Numerical data input
        // byte, short and float can be read
        // using similar-named functions.
        int age = sc.nextInt();

        long mobileNo = sc.nextLong();

        double cgpa = sc.nextDouble();
```

```

// Print the values to check if the input was correctly obtained.

System.out.println("Name: "+name);

System.out.println("Gender: "+gender);

System.out.println("Age: "+age);

System.out.println("Mobile Number: "+mobileNo);

System.out.println("CGPA: "+cgpa);

}

}

```

Can we have more than one constructor in class? If yes, explain the need for such a situation. [3]

Yes, a Class in ABL can have more than Constructor.

Multiple instance constructors can be defined for a class that is overloaded with different parameter signatures. If an instance constructor is defined without parameters, that constructor becomes the default instance constructor for the class. Only one static constructor for a class can be defined.

ex: <https://knowledgebase.progress.com/articles/Article/Can-a-Class-have-more-than-one-Constructor>

Write a class definition for the following scenario. You have to write a class called MyFraction which works on fractions of the form a/b where a represents the numerator and b represents the denominator. Both a and b are integers (i.e. if a = 1 and b = 2. then the fraction will be 1/2). Your class should perform the following operations:

i. Create two constructors - (1) with no parameters that set a= 0 and b= 1; and (2) with 2 integer parameters that sets both a and b.

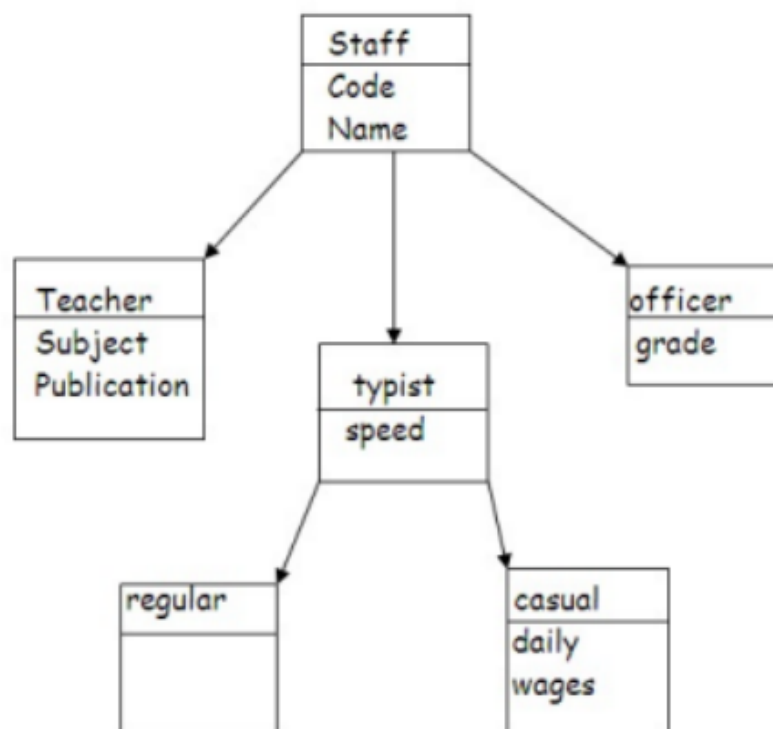
ii. Write an instance method for addition which returns the resultant object. [int: a/b + c/d = (ad + bc)/bd]

iii. Write a static method for multiplication which returns the resultant object. [Hint $a/b * c/d = ac/bd$]

Write a static main that allocates two Fractions, $1/2$ and $3/4$ and stores their sum in a third variable.

#

Barishal University expects to keep a database of its staffs. The database is separated into several classes whose hierarchical connections are shown in following figure. The figure also shows the minimum details required for each class. Specify all classes and define functions to create the database and retrieve individual information as and when required.



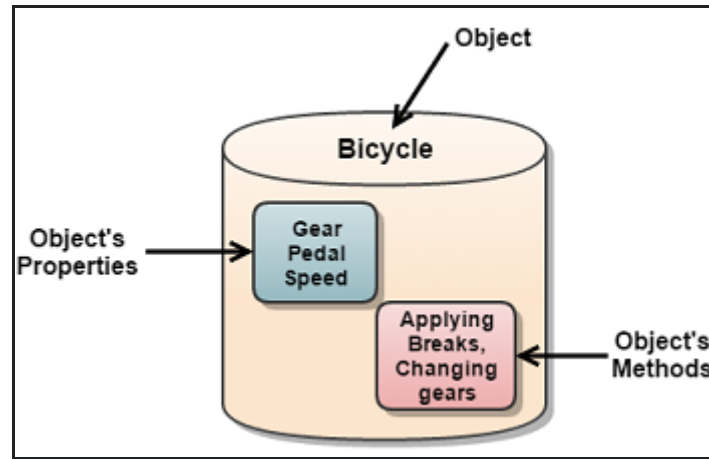
1
0

ans:

Object

What is object? Briefly explain with an example.

An object in OOP is nothing but a self-contained component that consists of methods and properties to make a particular type of data useful.



#> Dynamic binding uses which information for binding?

Ans: Object

Interface

What is the interface? Briefly explain with an example.

An interface is a reference type in Java. It is similar to class. It is a collection of abstract methods.

Or,

An Interface in Java programming language is defined as an abstract type used to specify the behavior of a class. An interface in Java is a blueprint of a class. A Java interface contains static constants and abstract methods.

Example:

```
public interface MyInterface {

    public String hello = "Hello";

    public void sayHello();
}
```

https://www.tutorialspoint.com/java/java_interfaces.htm

Details:

<https://www.javatpoint.com/interface-in-java>

<https://www.geeksforgeeks.org/interfaces-in-java/>

#> Describe the various forms of implementing interfaces with examples. [5]

<http://tutorials.jenkov.com/java/interfaces.html>

https://www.tutorialspoint.com/java/java_interfaces.htm

Method

What is method? Briefly explain with an example.

A method in Java or Java Method is a collection of statements that perform some specific task and return the result to the caller.

A Java method can perform some specific task without returning anything. Methods in Java allow us to reuse the code without retyping the code

Create a Method

A method must be declared within a class. It is defined with the name of the method, followed by parentheses (). Java provides some pre-defined methods, such as `System.out.println()`, but you can also create your own methods to perform certain actions:

Example

Create a method inside Main:

```
public class Main {
    static void myMethod() {
        // code to be executed
    }
}
```

```
}
```


Illustrate static method with proper example.

Static Method

Static methods are the methods in Java that can be called without creating an object of class. They are referenced by the class name itself or reference to the Object of that class.

```
public static void geek(String name)
{
    // code to be executed....
}

// Must have static modifier in their declaration.
// Return type can be int, float, String or user defined data
type.
```

##What if main method is not public static in Java? [4]

1. Since the main method is static Java Virtual Machine can call it without creating an instance of a class that contains the main method.
2. Since C and C++ also have a similar main method which serves as an entry point for program execution, following that convention will only help Java.
3. If the main method were not declared static then JVM has to create an instance of the main Class and since the constructor can be overloaded and can have arguments there would not be any certain and consistent way for JVM to find the main method in Java.
4. Anything which is declared in class in Java comes under reference type and requires objects to be created before using them but the static method and static data are loaded into separate memory inside JVM called context which is created when a class is loaded. If the main method is static then it will be loaded in the JVM context and are available for execution.

Read more:

<https://javarevisited.blogspot.com/2011/12/main-public-static-java-void-method-why.html#ixzz7PIBmlzYn>

<https://javarevisited.blogspot.com/2011/12/main-public-static-java-void-method-why.html#axzz7PDylLAfr>

Why a class might provide a set method and a get method for an instance variable. Explain with an example. [4]

A class might provide a set method (setter) and a get method (getter) for an instance variable in order to encapsulate the internal implementation of the class, and to isolate that implementation from the public interface of the class. Even if the setter and getter does no more than directly set and get the instance variable, the public interface should use that paradigm in order to allow for possible future alteration of the implementation, while minimizing the impact of doing so. Proper OOD/P (Object Oriented Design/Programming) should always make the implementation be private, even for derived classes. This reduces error, and reduces "lazy" coding techniques.

What is the difference between method overloading and method overriding? [3]

<https://www.geeksforgeeks.org/difference-between-method-overloading-and-method-overriding-in-python/>

#> Which of the following concepts means determining at runtime what method to invoke?

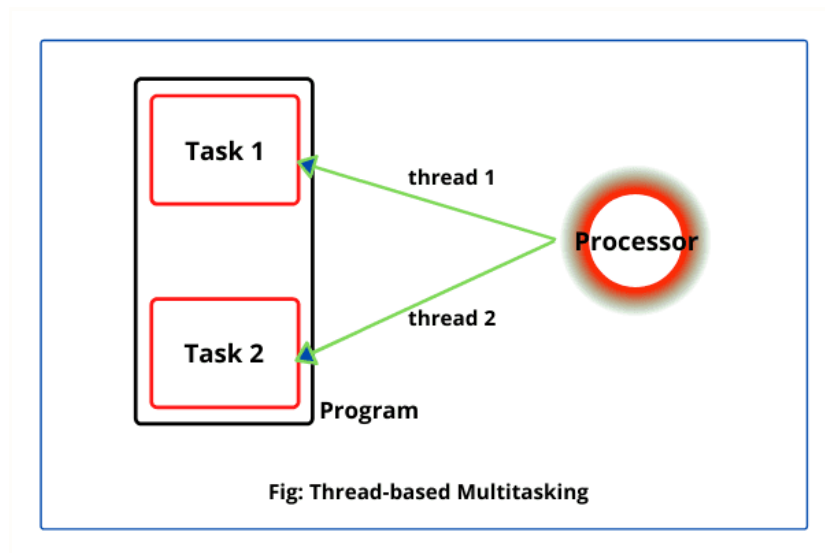
Ans: Dynamic Binding

Thread

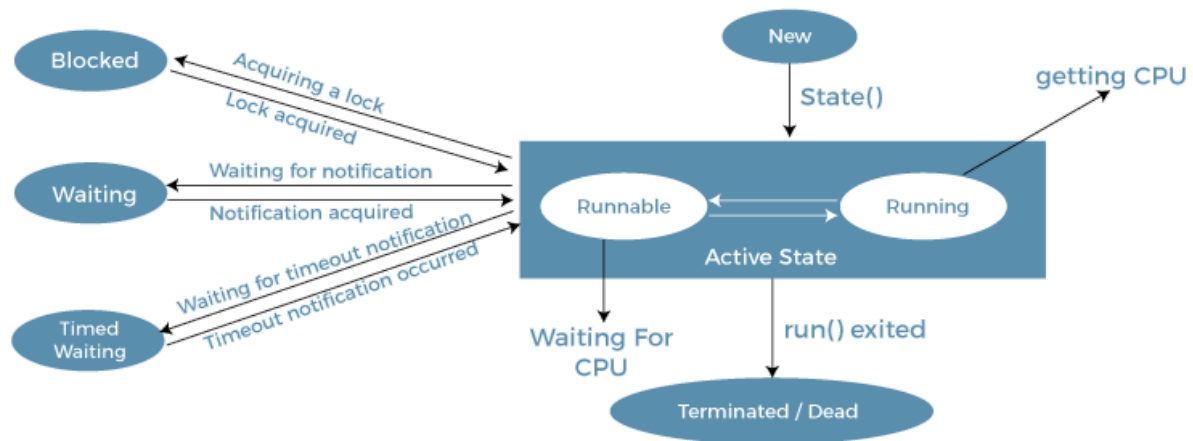
What is thread? Briefly explain with an example.

Threads allows a program to operate more efficiently by doing multiple things at the same time.

A thread is a thread of execution in a program. The Java Virtual Machine allows an application to have multiple threads of execution running concurrently. Every thread has a priority. Threads with higher priority are executed in preference to threads with lower priority.

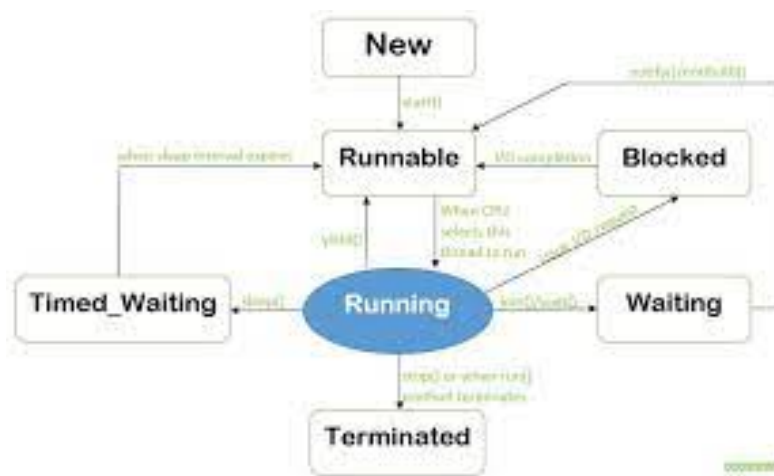


Draw the complete life cycle of a thread. [2]



Life Cycle of a Thread

or,

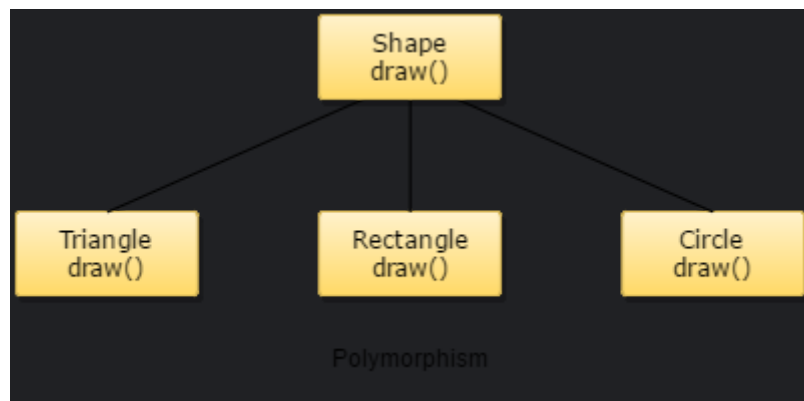


Polymorphism

What is polymorphism? Briefly explain with an example

Polymorphism is one of the OOPs feature that allows us to perform a single action in different ways.

Polymorphism is the ability of an object to take on many forms. The most common use of polymorphism in OOP occurs when a parent class reference is used to refer to a child class object. Any Java object that can pass more than one IS-A test is considered to be polymorphic.



#> How many types of polymorphisms?

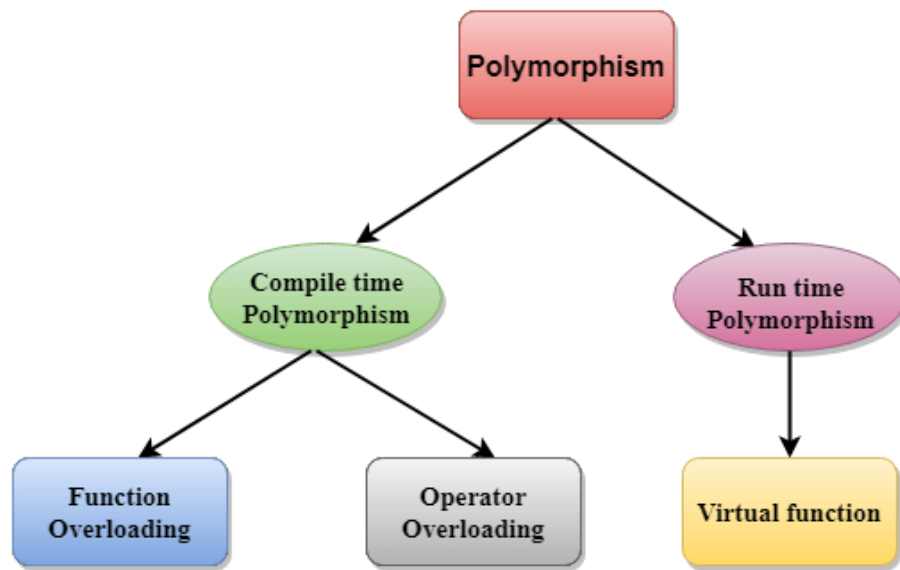
Ans:

In Object-Oriented Programming (OOPS) language, there are two types of polymorphism as below:

Static Binding (or Compile time) Polymorphism,

e.g., Method Overloading

Dynamic Binding (or Runtime) Polymorphism, e.g., Method overriding



Jodi beshi marks er dey tar jonno nicer link :

<https://www.upgrad.com/blog/polymorphism-in-oops/#:~:text=being%20used%20widely,-,Types%20of%20Polymorphism%20in%20Ops,-In%20Object%2DOriented>

#> The ability of a function or operator to act in different ways on different data types is called ____.

Ans:

Ans: Polymorphism

Polymorphism

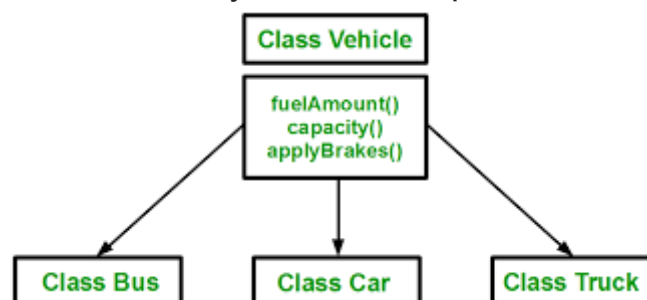
- Polymorphism is the approach of achieving the same action in multiple ways.
- Polymorphism means "many forms," which occurs when there are several classes that are connected via inheritance. We can inherit properties and methods from another class using inheritance.
- Polymorphism uses these techniques for a variety of purposes. This enables us to carry out a single activity in a variety of ways.

Inheritance

What is inheritance? Briefly explain with an example.

Inheritance is a mechanism in which one class acquires the property of another class.

For example, a child inherits the traits of his/her parents. With inheritance, we can reuse the fields and methods of the existing class. Hence, inheritance facilitates Reusability and is an important concept of OOPs.



Details:

<https://www.guru99.com/java-class-inheritance.html#:~:text=Inheritance%20is%20a%20mechanism%20in,methods%20of%20the%20existing%20class>

**ei link er super keyword er age porjonto dekhbi....

What do you know about inheritance in java? What are the advantages of inheritance? [4]

Prothm qsn er answer ager qsn er link ta...

Advantages:

- Inheritance promotes **reusability**. When a class inherits or derives another class, it can access all the functionality of inherited class.
- Reusability enhanced **reliability** (নির্ভরযোগ্যতা). The base class code will be already tested and debugged.
- As the existing code is reused, it leads to **less development and maintenance costs**.
- Inheritance makes the sub classes follow a standard interface.
- Inheritance helps to **reduce code redundancy** and supports code extensibility.
- Inheritance facilitates creation of class libraries.

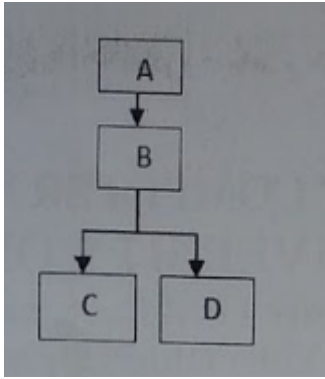
Disadvantages:

- Inherited functions **work slower** than normal function as there is indirection.
- Improper use of inheritance may lead to **wrong solutions**.
- Often, data members in the base class are left unused which may lead to memory wastage.
- Inheritance increases the coupling between base class and derived class. A **change in base class will affect all the child classes**.

#> The process of building new classes from existing ones is called ____

Ans: Inheritance

Show how the following inheritance hierarchy can be implemented in java? [3]py
ny re answer....



Does Java support multiple inheritances? Justify your answer. [2]

When one class extends more than one classes then this is called multiple inheritance. For example: Class C extends class A and B then this **type of inheritance** is known as multiple inheritance. **Java doesn't allow multiple inheritance.** C++ , Common lisp and few other languages supports multiple inheritance while java doesn't support it. Java doesn't allow multiple inheritance to avoid the ambiguity (অস্পষ্টতা) caused by it. One of the example of such problem is the diamond problem that occurs in multiple inheritance.

For details:

<https://beginnersbook.com/2013/05/java-multiple-inheritance/#:~:text=C%2B%2B%20%2C%20Common%20lisp%20and%20few,the%20ambiguity%20caused%20by%20it>

What happens when a protected member is inherited as public? What happens when it is inherited as private? What happens when it is inherited as protected? [3]

When using private inheritance, all public and protected members of the base class will become private members in the derived class. Only member functions of the derived class can access them. Moreover, it is a stop to further inheritance, as private members don't get further inherited

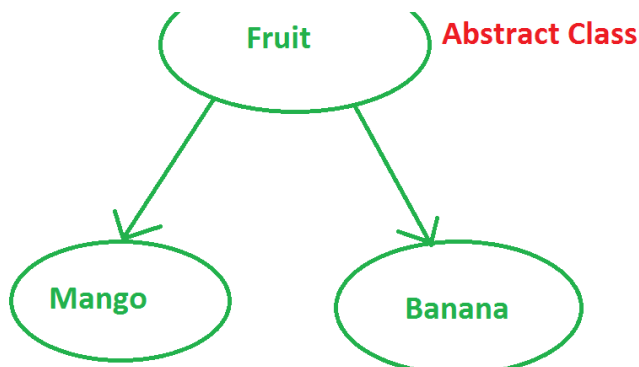
When using protected, your public and protected members of base class become protected members in derived class. It isn't a stop to inheritance as you can go further.

Abstraction

What is abstraction? Briefly explain with an example. [Nayeem]

Abstraction is a technique of hiding unnecessary details from the user.

Data Abstraction is defined as the process of reducing the object to its essence so that only the necessary characteristics are exposed to the users.



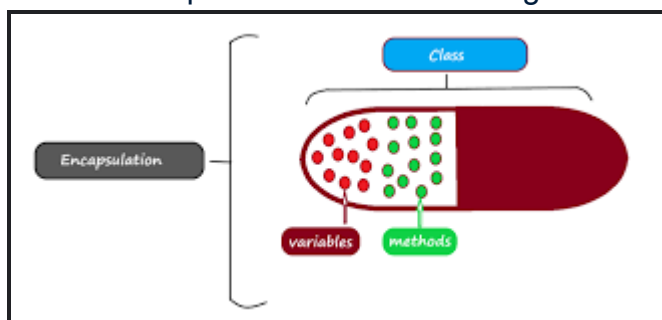
#> Which of the following concepts of OOPS means exposing only necessary information to the client?

Ans: abstraction

Encapsulation

What is encapsulation? Briefly explain with an example.

Encapsulation refers to the bundling of fields and methods inside a single class. It prevents outer classes from accessing and changing fields and methods of a class. This also helps to achieve data hiding.



#> Which of the following concepts means wrapping up data and functions together?

Ans: Encapsulation

Questions from Code

#

4. What will be the Output of the below code? Write Answer with explanation.	
i) <pre> public class A { public static void main(String[] args) { if (true) break; } } </pre>	Choice: i) Nothing ii) Error Answer: Reason:
ii) <pre> public class A { public static void main(String[] args) { System.out.println('j' + 'a' + 'v' + 'a'); } } </pre>	Choice: i) java ii) Something else (Other than simple concatenation) Answer: Reason:
iii) <pre> public class A { public static void main(String[] args) { int \$_ = 5; } } </pre>	Choice: i) Nothing ii) Error Answer: Reason:
iv) <pre> public class Demo{ public static void main(String[] arr){ Integer num1 = 100; Integer num2 = 100; Integer num3 = 500; Integer num4 = 500; if(num1==num2){ System.out.println("num1 == num2"); } else{ System.out.println("num1 != num2"); } if(num3 == num4){ System.out.println("num3 == num4"); } else{ System.out.println("num3 != num4"); } } } </pre>	Choice: i) num1 == num2 num3 == num4 ii) num1 == num2 num3 != num4 iii) num1 != num2 num3 == num4 iv) num1 != num2 num3 != num4 Answer: Reason:

- i) Error
 - ii) Something else
 - iii) Nothing
 - iv) num1==num2 , num3 != num4
- Details coming soon...

##

What is Dangling-else problem? Determine the output for the given set of code when x is 9 and y is 11 and when x is 11 and y is 9.

```
if(x<10)
if(y>10)
System.out.println("*****");
else
System.out.println("#####");
System.out.println("$$$$$");
```

The dangling else problem is syntactic ambiguity. It occurs when we use nested if. When there are multiple “if” statements, the “else” part doesn’t get a clear view with which “if” it should combine.

For example:

```
if (condition) {
}
if (condition 1) {
}
if (condition 2) {
}
else
{
}
```

In the above example, there are multiple “ifs” with multiple conditions and here we want to pair the outermost if with the else part. But the else part doesn’t get a clear view with which “if” condition it should pair. This leads to inappropriate results in programming.

Ref:

<https://googler700.blogspot.com/2015/03/423-c-how-to-program-8th-edition-by.html>

(Dangling Else Problem) State the output for each of the following when x is 9 and y is 11 and when x is 11 and y is 9.

a)

```
if ( x < 10 )
```

```
if ( y > 10 )
```

```
cout << "*****" << endl;
```

```
else
```

```
cout << "#####" << endl;
```

```
cout << "$$$$$" << endl;
```

b)

```
if ( x < 10 )
```

```
{if ( y > 10 )
```

```
cout << "*****" << endl;
```

```
}else
```

```
{
```

```
cout << "#####" << endl;
```

```
cout << "$$$$$" << endl;
```

```
}
```

Solution:

a)

```
x = 9, y = 11
```

```
*****
```

\$\$\$\$\$

x = 1 1 , y = 9

\$\$\$\$\$

b)

x = 9, y = 1 1

x = 1 1 , y = 99

#####

\$\$\$\$\$

##

Write a Java program that will perform the following operations: 3

- Create an object of type ArrayList that will contain a list of floating-point numbers.
- Now insert the following data: 12.34, 34.5, 5.6, 7.89, 10.12, 3.45
- Show the number of elements in the object.
- Remove 5.6 and 10.12

Display the content of the object. 3

collection and finalize method? Explain briefly.

##

Write a program to create a Database of the employees of an Organization with Name, Designation, Experience, Birth Date, Salary and Address information where Name, Designation and Experience are public members, Birth Date private member and other information are protected members. 6

ans—>

##

Consider a class named **BankAccount** that has three methods with preconditions: 6

1. The constructor (initial balance must not be negative)
2. The withdraw method (withdrawal amount must not be less than the balance)
3. The deposit method (deposit amount must not be negative)

Write code for the **BankAccount** class so that each of the three methods throws an **IllegalArgumentException** if the precondition is violated.

<http://users.cis.fiu.edu/~shawg/2210/BankAccountTest2.java>

(Not exactly same.... But will get some idea)

##

Consider the following code segment:

```
public interface Red {
    public void turnRed()
}
public interface Blue {
    public void turnBlue()
}
```

```
public class A implements Red {
    public void one(){
        System.out.println("A.1");
    }
    public void two(){
        one();
        System.out.println("A.2");
    }
    public void turnRed(){
        System.out.println("Noooo!!");
    }
}
```

```
public class B extends A implements Red,
Blue {
    public void one(int x){
        System.out.println("B.1 "+x);
    }
    public void two(){
        System.out.println("B.2");
    }
    public void turnBlue(){
        System.out.println("Aaieee!!");
    }
}
```

```
public class C extends A {
    public void one(){
        System.out.println("C.1");
    }
}
```

For each of the following, determine the output when the code will be executed. If some sort of errors are generated, give a brief explanation of the problem.

(a) `A a = new B();`
`a.two();`

(b) `B b = new B();`
`b.one(); b.one();`

(c) `Red r = new B();`
`r.turnRed();`

(d) `Blue b = new Blue();`
`b.turnBlue();`

(e) `C c = new A();`
`c.two();`

(f) `A a = new B();`
`a.turnBlue();`

(g) `A a = new C();`
`a.two();`

Block

What is block? Briefly explain with an example.

ANS:

A **block in Java** is a set of code enclosed within curly braces { } within any class, method, or **constructor**. It begins with an opening brace ({) and ends with an closing braces (}).

Between the opening and closing braces, we can write codes which may be a group of one or more statements.

The syntax for a set of code written in a block of a class is as follows:

Syntax:

```
public class Student
{ // block start.
    // class body
} // block end.
```

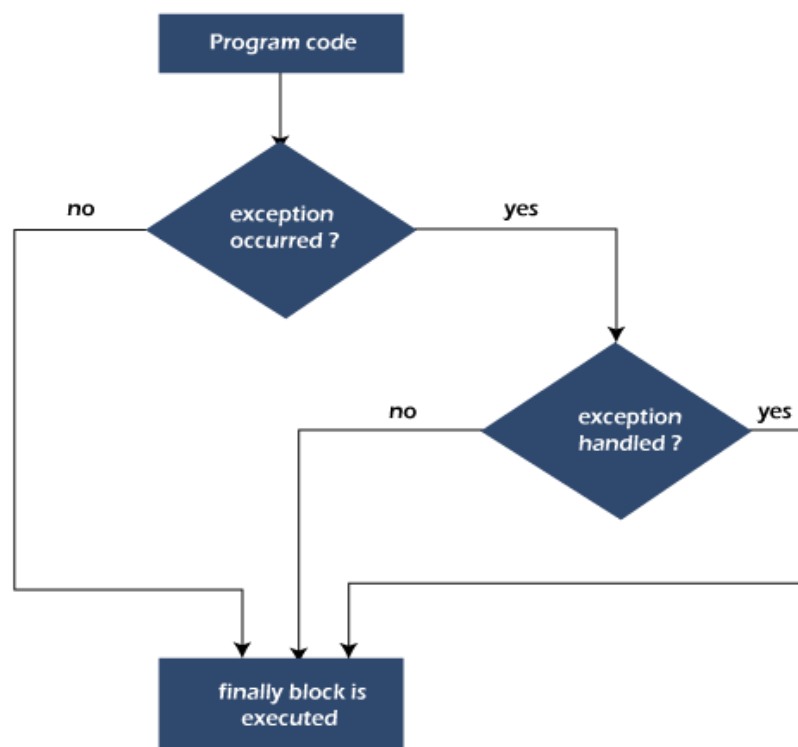
What is a finally block? When and how is it used? Give a suitable example. [3]

Ans:

Java finally block is a block used to execute important code such as closing the connection, etc.

Java finally block is always executed whether an exception is handled or not. Therefore, it contains all the necessary statements that need to be printed regardless of the exception occurs or not.

The finally block follows the try-catch block.



Why use Java finally block?

- finally block in Java can be used to put "cleanup" code such as closing a file, closing connection, etc.

- The important statements to be printed can be placed in the finally block.

Create a try block that is likely to generate three types of exception and then incorporate necessary catch blocks to catch and handle them appropriately. [3]

ANS:

```
public class Test {

    public static void main (String args []){

        try{

            int a = 1/0;
        }

        catch (ArithmeticException e){

            System.out.println (e);

        }
    }
}
```

What is an Exception? Explain the syntax of try block and catch block with an example. [3]

ANS:

The term exception is shorthand for the phrase "exceptional event." Definition: An exception is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.

Java **try** block is used to enclose the code that might throw an exception. It must be used within the method.

If an exception occurs at the particular statement in the try block, the rest of the block code will not execute. So, it is recommended not to keep the code in try block that will not throw an exception.

Java try block must be followed by either catch or finally block.

Syntax of Java try-catch

1. **try**{

2. `//code that may throw an exception`
3. `}catch(Exception_class_Name ref){}`

Syntax of try-finally block

1. `try{`
2. `//code that may throw an exception`
3. `}finally{}`

Explain the use of finally block. [3]

ANS:

A “finally” is a keyword used to create a block of code that follows a try or catch block. A finally block contains all the crucial codes such as closing connections, stream, etc that is always executed whether an exception occurs within a try block or not.

When finally block is attached with a *try-catch block*, it is always executed whether the catch block has handled the exception thrown by try block or not.

The syntax for try-finally and try-catch-finally is as follows:

Syntax for try-finally block:

```
try
{
    statement1;
    statement2;
}
finally // finally block
{
    statement3;
}
```

Syntax for try-catch-finally block:

```
try
{
    statement1;
    statement2;
}
catch(Exceptiontype e1)
{
```

```

    statement3;
}
statement4;
finally
{
    statement5;
}

```

Some important rules of using finally block or clause are:

1. A finally block is optional but at least one of the catch or finally block must exist with a try.
2. It must be defined at the end of last catch block. If finally block is defined before a catch block, the program will not compile successfully.
3. Unlike catch, multiple finally blocks cannot be declared with a single try block. That is there can be only one finally clause with a single try block.

Variable & Data Type

What is static variable? [1]

Static variable in Java is variable which belongs to the class and initialized only once at the start of the execution. It is a variable which belongs to the class and not to object(instance). Static variables are initialized only once, at the start of the execution.

#> Size of double data type is ____. Ans: 64

#> What is a data type?

Ans: A particular scheme for representing values with bit patterns.

#> A variable is a named _____ which contains a value. Ans: memory location

#> Converting several data types into one single one is called _____ Ans: type conversion.

#> Java assumes fractional numbers to be of _____ data type. Ans: double

#> Which of the following is NOT the name of a Java primitive data type? Ans:

String

#> Number of primitive data types in java? Ans: 8

#> Size of boolean data type is ____ Ans: 1 bit

#> In Java byte, short, int, and long all of these are

both

none

signed

Unsigned

(শিউর বা)

#> _____ is the functional and fundamental unit of a computer program. Ans:

Token

#> Implicit type conversion is also known as ____ Ans: coercion

#> The following algorithms each use data values. Can you work out which of these is using fixed values and which is using variable values?

Algorithm A

```
Add 4 to 5  
Print the result
```

Algorithm B

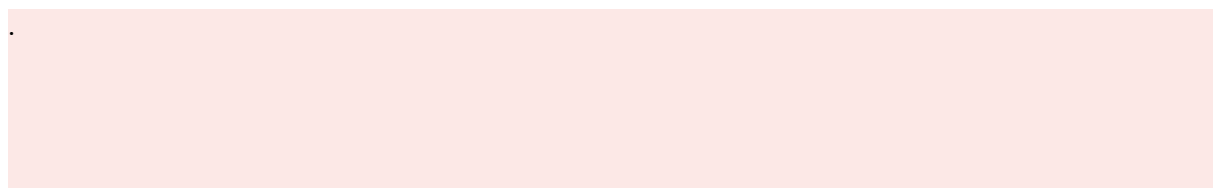
```
Ask the user to type in a whole number  
Add 5 to that number  
Print the result
```

Algorithm C

```
If 4 is less than 5  
    Add 5 to 4  
    Print the result  
Otherwise  
    Print "no"
```

Algorithm D

```
Ask the user to type in a whole number  
If the number typed in is less than 5  
    Add 5 to the number typed in  
    Print the result  
Otherwise  
    Print "no"
```



Correct answers

Algorithms A and C are using fixed values (4 and 5). There is no option within the algorithm to alter the data. These algorithms will always produce the same results.

Algorithms B and D are using variable data. The data input depends on the choice of the user. The algorithm will output a different result depending on the user input.